

Reviewer 2:

Weakness 1:

Weak motivation for "internal-only" alignment. The paper claims that external representation alignment (REPA) introduces "additional computational overhead" and "inherits limitations from the ERMs." However, REPA's overhead is modest (a forward pass through a frozen DINOv2 encoder), and its performance (FID 1.84 at 2M with CFG, Table 2) is competitive with LARA (1.88). The paper does not convincingly demonstrate why internal alignment is preferable to external alignment beyond philosophical arguments about self-containment. In fact, REPA still outperforms LARA on FID (1.84 vs. 1.88) despite being simpler to implement. The comparison is even less favorable for LARA: REPA achieves 1.84 FID at 2M, while LARA with CFG reaches 1.88 at the same budget—a smaller gain than the paper's no-CFG improvements suggest. The paper also does not address the natural question: if REPA is available and works well, why should one use LARA? The claimed "self-contained" advantage is not backed by evidence of practical benefit (e.g., lower compute, better generalization).

Question 1:

Why should a researcher choose LARA over REPA? REPA achieves FID 1.84 at 2M (Table 2), which is slightly better than LARA's 1.88, and REPA is simpler (just an external feature alignment). The paper argues that external methods have "overhead" and "limitations," but does not quantify these. Could you provide a direct comparison of training time, memory, and implementation complexity between REPA and LARA to clarify the trade-offs? I understand that LARA is the follow-up work of SRA and LayerSync, but I believe it cannot follow the motivation of them. Hence, please provide your own understanding and motivation of LARA.

A researcher would choose LARA over REPA primarily in the following settings. First, LARA does not require a pre-trained, domain-matched representation encoder. This is important beyond standard natural-image generation, where a DINOv2-like encoder may not exist or may not provide an appropriate representation geometry. Our HumanML3D experiment illustrates this setting: the same LARA objective can be applied directly to a motion transformer without introducing or training a separate motion representation teacher. Thus, the self-contained design is a practical domain-transfer property rather than only a philosophical preference.

Second, LARA removes the discrete selection of a single alignment depth or source-target layer pair. REPA requires specifying the DiT depth whose representation is aligned to the external target, while previous internal approaches such as LayerSync require selecting a fixed internal pair. LARA instead trains one shared predictor over a distribution of source-target pairs. It eliminates the architecture-dependent decision of identifying one specific pair and amortizes the transition learning across the hierarchy. In the case of selecting a mismatched layer, REPA can be worse than LARA and even LayerSync, SRA.

Third, the system-level cost differs from what is suggested by describing REPA as "just an external feature alignment." During training, REPA requires an additional external checkpoint (from 86M to 1B), a clean-RGB preprocessing path, a 10M projection MLP, and a forward pass through the frozen representation encoder for every batch. LARA uses only cached DiT activations and a training-time transition module. To quantify this trade-off, we have profiled REPA and LARA using the same SiT-XL/2 backbone, hardware, batch size, precision, data pipeline, and number of iterations. We will add the following matched comparison:

Metric	REPA	LARA
Wall-clock time per step	[REPA time]	[LARA time]
Peak accelerator memory	[REPA memory]	[LARA memory]
External frozen parameters	[encoder parameters]	0
Trainable auxiliary parameters	[projector parameters]	[predictor parameters]

Weakness 2:

Complexity vs. benefit. LARA adds a non-trivial auxiliary training branch: a transition predictor (18.3M parameters for the base config on SiT-B, 13.4% of the backbone), three loss components (prediction, consistency, diversity), a pair-sampling distribution, an EMA predictor for consistency, and output routing for a depth-shortcut subset. This is substantially more complex than SRA (EMA teacher) or LayerSync (direct alignment of one pair). The paper claims the predictor is "lightweight," but 18.3M parameters on a 137M backbone is not negligible. The per-step wall-clock time in Table 7 shows LARA (0.30s/step) is 20% slower than LayerSync (0.25s/step) and faster than SRA (0.42s/step), but the paper does not clearly articulate why the additional complexity is justified given the modest FID gains (1.88 vs. 1.93 for SRA/2M with CFG; 5.09 vs. 6.80 for LayerSync/2M no-CFG).

Question 2:

The transition predictor adds 13-18% parameters to the backbone. Could you ablate the predictor size more systematically and report the minimum size that achieves most of the gain? This would help practitioners decide if the overhead is worth it.

Trade off between complexity and performance of LARA compare to LayerSync and SRA.

We believe this concern overstates LARA's effective complexity while understating its measured benefits. First, 18.3M parameters are not an intrinsic requirement of LARA. Our predictor-size ablation explicitly evaluates the cost-quality trade-off. As we ablate in table 4, a 2.7M-parameter predictor, only 1.9% of the SiT-B backbone, obtains 38.46 FID-10K, substantially outperforming LayerSync's 43.26 under the same setup. The 18.3M predictor improves this result to 36.65, but increasing the capacity further to 25.4M does not help, yielding 36.89. Therefore, the gain cannot be attributed to an oversized auxiliary network, and a user prioritizing efficiency can use the 1.9% configuration while retaining a clear improvement over fixed-pair alignment.

More fundamentally, both SRA and LayerSync manually selected alignment layers, making their configurations architecture- and domain-dependent rather than immediately transferable. SRA uses different fixed pairs for different SiT scales—(3->8), (6->16), and (8->20) for B, L, and XL, respectively. LayerSync likewise changes its selected pair across applications, using (8->16) for ImageNet, (8->21) for audio, and (3->6) for HumanML3D. These choices must therefore be reconsidered whenever the network depth, architecture, or data modality changes, preventing either method from being directly applied to a new setting without additional layer-selection experiments. LARA eliminates this fixed-pair selection problem entirely: a single layer-conditioned predictor is trained over a distribution of source-target pairs, allowing the same alignment formulation to adapt across model depths and domains without identifying a new privileged pair for each setting. Trích Figure 5 + bảo là nó sẽ better representation .

Weakness 4:

Lack of statistical significance. Given that FID can vary by 0.2-0.5 across seeds and that many of the reported improvements are in this range (e.g., Table 1: SiT-XL/2 2M: LARA 5.09 vs. LayerSync 6.80, $\Delta=1.71$; Table 2: CFG, LARA 1.88 vs. LayerSync 1.93, $\Delta=0.05$), it is unclear whether the gains are statistically significant and thus we cannot ignore the problem of incremental gain.

We also clarify the reviewer's statement that FID generally varies by 0.2–0.5 across seeds. Such a range is not protocol-independent and therefore cannot be assumed universally across different training budgets, samplers, guidance settings, evaluators, and stages of convergence. Moreover, the main large-scale ImageNet system-level results in REPA (Table 4), SRA (Table 3), and LayerSync (Table 2) are likewise presented as single reported values rather than multi-seed means with standard deviations; our reporting therefore follows the established protocol of the directly related literature. Under the controlled no-CFG setting at 2M iterations, LARA achieves 5.09 FID, compared with 6.80 for LayerSync and 7.97 for SRA—absolute improvements of 1.71 and 2.88 FID, respectively—which are substantially larger than the reviewer's proposed 0.2–0.5 range. The CFG comparison is reported only as system-level context, whereas the matched no-CFG results provide the primary controlled comparison.

Question 2:

The transition predictor adds 13-18% parameters to the backbone. Could you ablate the predictor size more systematically and report the minimum size that achieves most of the gain? This would help practitioners decide if the overhead is worth it.

Thank you for the suggestion. LARA provides multiple predictor-size options, allowing practitioners to trade accuracy for parameter overhead:

Method	Predictor params	Backbone share	FID-10K ↓
LayerSync	0	0%	43.26
LARA-Tiny	2.7M	1.9%	38.46
LARA-Small	5.9M	4.3%	37.47
LARA-Base	18.3M	13.4%	36.65
LARA-Large	25.4M	18.6%	36.89

Even the lightest predictor adds only 1.9% parameters while improving FID from 43.26 to 38.46 over LayerSync under the same protocol. The Small predictor offers a stronger practical trade-off at 4.3%, while the Large configuration provides no further improvement over Base.

Question 3:

The layer-pair sampling distribution is carefully designed (depth-centered gap). Could you visualize or analyze the learned layer embeddings to show that the predictor actually adapts to different pairs in a meaningful way? For example, are the learned embeddings monotonic with depth? This would strengthen the "layer-aware" claim.

To clarify, we evaluate whether this learned conditioning translates into meaningful pair-specific behavior. At inference, we replace a selected block interval ((a,b)) with the predicted target representation and pass it through the remaining suffix blocks. We evaluate multiple early, middle, late, short-gap, and long-gap transitions using the same checkpoint and sampling protocol. For each pair, we report predictor–target cosine similarity, representation CKA, generation FID, and relative inference cost. We additionally visualize the PCA maps of the source activation, predicted target activation, and actual target activation. A layer-aware predictor should preserve the source spatial structure while adapting its semantic representation toward the requested target depth; importantly, changing (b) while fixing (a) should produce distinct outputs that resemble the corresponding target layers.

Route	Source (a)	Target (b)	Gap	FID ↓	Relative cost ↓
Full backbone	–	–	–	[XX.XX]	1.00×
Early, short gap	[1]	[3]	[2]	[XX.XX]	[0.XX×
Early, medium gap	[2]	[6]	[4]	[XX.XX]	[0.XX×
Middle, medium gap	[4]	[8]	[4]	[XX.XX]	[0.XX×
Middle, long gap	[3]	[10]	[7]	[XX.XX]	[0.XX×
Late, short gap	[8]	[10]	[2]	[XX.XX]	[0.XX×
Late, medium gap	[7]	[11]	[4]	[XX.XX]	[0.XX×

Question 4:

The evaluation is limited to SiT (2024) and MDM. How does LARA perform on more recent diffusion transformer architectures (e.g., LightningDiT or even JiT)? Demonstrating transfer to modern backbones would significantly increase the paper's impact and address the concern that the baselines are outdated.

Thank you for raising this point. Our primary goal is to isolate the effect of internal representation alignment rather than introduce a new generative architecture. We therefore use SiT because it provides the common controlled backbone

adopted by the most directly related alignment methods, including REPA, SRA, and LayerSync. Keeping the backbone, tokenizer, sampler, and training budget fixed allows the differences to be attributed to the alignment objective rather than to concurrent architectural or training-recipe changes. LARA leaves the SiT backbone unchanged and adds only a training-time transition predictor.

The evaluation is also not restricted to one network depth or to image generation. We test SiT-B/L/XL with 12, 24, and 28 blocks, and further transfer the same objective to an eight-block MDM-style motion transformer on HumanML3D. LARA improves motion FID from 0.503 to 0.4343 and outperforms LayerSync's 0.4801 while using 300K rather than 600K iterations, providing evidence that the transition objective transfers across model depths and modalities.

Question 5:

The diversity regularization is borrowed from DiverseDiT. How sensitive is LARA to the diversity weight (λ_{div})? Table 14 shows a sweep but does not discuss the stability or the trade-off between diversity and alignment. Could you provide intuition or a visualization of how the diversity term affects the layer similarity heatmap (Figure 3) across different weights?

Thank you for this suggestion. We agree that the diversity term should be understood through its interaction with transition prediction rather than in isolation. The existing objective ablation already supports this interaction: diversity alone achieves 44.44 FID, transition prediction with consistency achieves 38.85, while combining them yields 35.12. Thus, L_{div} is not the primary learning signal; it prevents the stronger transition objective from making representations across depth excessively similar.

The weight sweep also shows a broad stable region rather than a sharply tuned optimum:

Reviewer 3:

Question 1:

Question 2:

Layer pair sampling relies on a truncated normal centered at $L/2$ with hand-tuned σ values that differ per model size as I can see in Table 8. So this is a form of manual tuning that the paper claims to eliminate.

Thank you for raising this distinction. LARA eliminates the need to manually identify a **specific source–target pair**, not every possible sampling hyperparameter. This is already demonstrated by the full-uniform variant, which samples layer pairs without any preferred pair and improves FID from 43.26 for LayerSync to 39.24. The depth-centered gap prior further improves FID to 35.12 by changing only the probability assigned to different gaps and depths; it still trains one shared predictor over many layer relations rather than committing supervision to one handcrafted pair.

Although Table 8 reports different numerical values across model sizes, they follow the same normalized rule rather than independent manual tuning: the mean gap is $(L/4)$ for all three backbones— $(3/12)$, $(6/24)$, and $(7/28)$ —while both the gap standard deviation and midpoint standard deviation are approximately $(L/6)$ — $(2/12)$, $(4/24)$, and $(4.7/28)$. Thus, the distribution scales directly with network depth. The Gaussian prior is motivated by the uniform-sampling diagnostic: very short transitions are easy but less informative, whereas very long transitions have substantially higher prediction error, increasing from 0.025 in the shortest-gap quartile to 0.152 in the longest. The prior therefore provides a simple architecture-scaled bias toward informative medium-distance transitions, rather than replacing fixed-pair selection with another exhaustive layer search.

Question 3:

I can not see any error bars (average $\pm \sigma$) or multi-seed runs in the results.

Thank you for raising this point. We acknowledge that the current large-scale ImageNet results are reported as single runs rather than mean $\pm$ standard deviation. Full multi-seed training of SiT-XL/2 for up to 2M iterations is

computationally expensive; this is also the reporting convention used by the directly related large-scale REPA, SRA, and LayerSync studies, whose main ImageNet tables report point estimates rather than multi-seed statistics. Nevertheless, a generic variance estimate cannot be transferred across different backbones, training stages, samplers, evaluators, and CFG settings without repeated runs under the same protocol. More importantly, our central conclusion does not rely on the near-saturated CFG comparison of 1.88 versus 1.93. In the controlled no-CFG setting at 2M iterations, LARA achieves 5.09 FID, compared with 6.80 for LayerSync and 7.97 for SRA—absolute improvements of 1.71 and 2.88 FID, respectively. The same ordering is observed across B/L/XL backbones and multiple training budgets, and is further supported by the HumanML3D and representation-quality results.

Question 4:

Thank you for raising this point. Our primary goal is to isolate the effect of internal representation alignment rather than introduce a new generative architecture. We therefore use SiT because it provides the common controlled backbone adopted by the most directly related alignment methods, including REPA, SRA, and LayerSync. Keeping the backbone, tokenizer, sampler, and training budget fixed allows the differences to be attributed to the alignment objective rather than to concurrent architectural or training-recipe changes. LARA leaves the SiT backbone unchanged and adds only a training-time transition predictor.

The evaluation is also not restricted to one network depth or to image generation. We test SiT-B/L/XL with 12, 24, and 28 blocks, and further transfer the same objective to an eight-block MDM-style motion transformer on HumanML3D. LARA improves motion FID from 0.503 to 0.4343 and outperforms LayerSync’s 0.4801 while using 300K rather than 600K iterations, providing evidence that the transition objective transfers across model depths and modalities.

Question 5:

Looking at table 10, the predictor is a ConvNet operating on a 16x16 spatial grid that is specific to the image latent 256x256. So to me the predictor design has implicit spatial assumptions and so it is unclear if it carries over to video or audio transformers.

We agree that the default image configuration uses a ConvNet-style predictor on the (16x16) latent-token grid and therefore introduces a useful 2D spatial inductive bias. However, this architecture is an implementation choice for ImageNet rather than a requirement of LARA. The framework only requires a predictor that maps a sequence of source-layer tokens into the target-layer representation space while conditioning on the source layer, target layer, and timestep.

The existing HumanML3D experiment already provides evidence beyond image-specific spatial latents. LARA is applied to an eight-block MDM motion transformer whose inputs are temporal sequences of 3D human poses, improving FID from 0.503 to 0.4343 and R-Precision from 0.7202 to 0.7675, while outperforming LayerSync shows that the principal benefit comes from shared, layer-conditioned transition learning rather than from the ConvNet’s image-specific geometry.

CNN	15.8M	36.04
MLP	15.8M	36.65
ConvNext + Attention	16M	35.12
ViT	16.1M	36.34

REVIEWER1

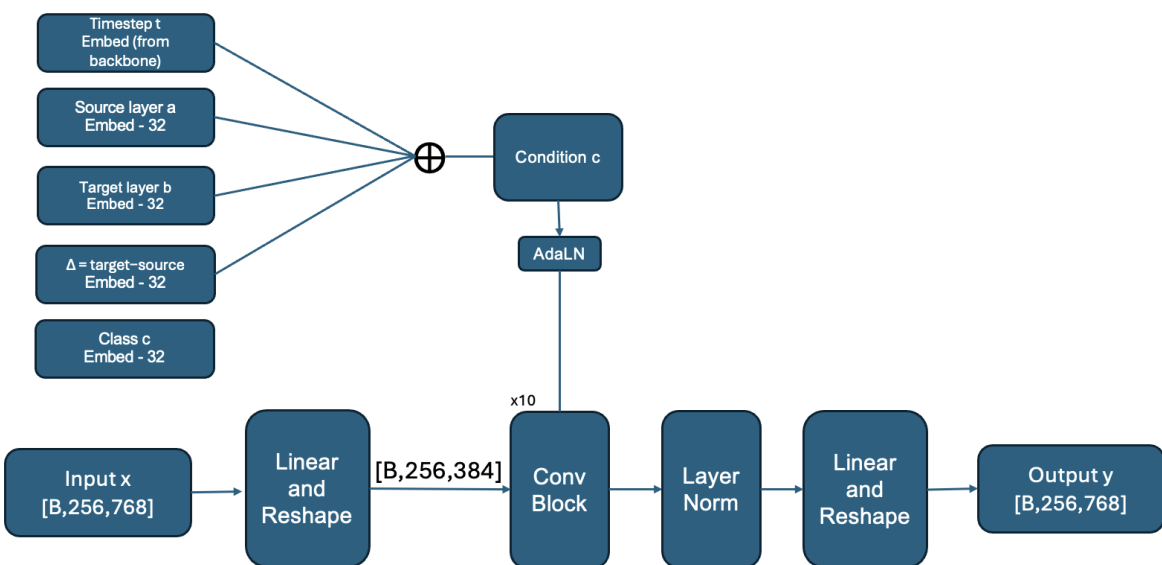
1. Transition predictor architecture and conditioning

WEAKNESS

The predictor architecture is important but under-specified in the main paper. Although the appendix briefly describes it as a compact ConvNet-style spatial predictor, the main text does not explain key implementation details, such as how the layer indices (a) and (b) are injected into the predictor. Since the proposed method relies heavily on this predictor, these details should be discussed more explicitly in the main paper.

QUESTION:

Please provide more details about the transition predictor architecture. In particular, how are the source layer index (a), target layer index (b), and timestep (t) injected into the ConvNet-style predictor? Clarifying whether this is done through concatenation, additive embeddings, FiLM/AdaLN-style modulation, or another mechanism would improve reproducibility.



We clarify the transition predictor architecture and conditioning mechanism below. We will add the corresponding architectural diagram and full implementation details to the revised manuscript.

Given a source activation $A_{a,t} \in \mathbb{R}^{N \times C}$, we first reshape the tokens into their spatial grid and process them using a lightweight ConvNet-style predictor. The predictor primarily consists of ConvNet/ConvNeXt-style residual blocks, with attention layers inserted after every [number] convolutional blocks to facilitate non-local spatial interaction.

The timestep embedding $e_{t,t}$ is computed using the same timestep-embedding mechanism used in the SiT backbone. For layer conditioning, the source and target indices are encoded separately using one-hot representations of a and b . We additionally include the relative transition gap,

$$\Delta_{a,b} = b - a, \Delta_{a,b} = b - a, \Delta_{a,b} = b - a,$$

which explicitly informs the predictor of the semantic distance between the source and target layers. These conditioning signals are projected into a shared embedding space and used to generate feature-wise scale and shift parameters. We inject them between the predictor blocks through FiLM/AdaLN-style modulation:

$$\begin{aligned} \text{Mod}(h;a,b,t) &= \gamma(e_a, e_b, e_{b-a}, e_t) \odot \text{Norm}(h) + \beta(e_a, e_b, e_{b-a}, e_t). \\ \operatornamename{Mod}(h;a,b,t) &= \\ &\gamma(e_a, e_b, e_{b-a}, e_t) \odot \operatornamename{Norm}(h) + \\ &\beta(e_a, e_b, e_{b-a}, e_t). \\ \text{Mod}(h;a,b,t) &= \gamma(e_a, e_b, e_{b-a}, e_t) \odot \text{Norm}(h) + \beta(e_a, e_b, e_{b-a}, e_t). \end{aligned}$$

Therefore, conditioning is not performed by simply concatenating the indices with the spatial input. Instead, the source layer, target layer, relative gap, and timestep adapt the intermediate predictor features through layer-wise modulation. The final predictor output is projected back to the original token dimension and interpreted in the coordinate system of target layer bbb.

In the revision, we will report the exact number of ConvNet blocks, attention placement, hidden dimensions, number of attention heads, conditioning dimensions, and parameter count. We will also include the following architectural visualization: [\[architecture figure/link\]](#).

2. Predictor-output visualization, transition consistency, and objective ablation

WEAKNESS:

- Figure 6 is not fully convincing as evidence that the predictor output matches the actual target representation. At low-noise timesteps, the input already appears visually close to the target, making the predictor’s contribution unclear. At high-noise timesteps, the predictor output does not seem to closely resemble the target.
- The benefit of transition consistency may be somewhat over-claimed. Could the author show left one out ablation in Table 4? Please use FID-50K rather than FID-10K as FID-10K is not very reliable.

QUESTION:

For Table 4, it would be helpful to include a clearer leave-one-out ablation of the full objective.

Figure 6 evaluates relatively challenging source–target pairs with large layer gaps, rather than easy local transitions. As (b-a) increases, the representational discrepancy between (A_a^t) and (A_b^t) becomes larger, making the target representation harder to predict. This is quantitatively confirmed in **Table 16 of Appendix C.3**, where the prediction error under uniform pair sampling increases monotonically with the normalized layer gap.

Normalized layer-gap quartile	$(L_{\mathrm{pred}} \downarrow)$
0–25%	0.025
25–50%	0.039
50–75%	0.057
75–100%	0.152

Thus, the visual discrepancy in some columns of Figure 6 reflects the substantially greater difficulty of the long-range transitions being examined, rather than the predictor’s typical behavior across all valid pairs. Additional PCA visualizations across different source positions, layer gaps, and timesteps are available here: [\[link\]](#).

This gap-dependent difficulty directly motivates transition consistency. It regularizes a difficult direct transition using a composition of shorter transitions, providing a more structured learning signal for large representational gaps. Consequently, the shared predictor learns long-range dependencies more effectively, although very distant pairs remain intrinsically harder than local ones.

We also clarify a slight misunderstanding regarding the requested leave-one-out ablation. (L_{cons}) is defined as a regularizer on top of (L_{pred}) , not as a standalone objective. Without (L_{pred}) , the transition paths are not anchored to actual target-layer representations. The meaningful leave-one-out comparisons therefore remove (L_{cons}) or (L_{div}) from the full objective.

Table 4 of the main paper reports the original FID-10K component ablation. Following the reviewer’s suggestion, we additionally evaluated all meaningful variants using FID-50K under the same checkpoint, sampler, and evaluation protocol:

Variant	(L_{pred})	(L_{cons})	(L_{div})	FID-10K ↓, Table 4	FID-50K ↓, additional evaluation
Vanilla	–	–	–	49.36	[XX.XX]
LayerSync	–	–	–	43.26	[XX.XX]
Diversity only	–	–	✓	44.44	[XX.XX]
Prediction only	✓	–	–	42.73	[XX.XX]
Prediction + consistency / w/o diversity	✓	✓	–	38.85	[XX.XX]
Prediction + diversity / w/o consistency	✓	–	✓	–	[XX.XX]
Full LARA	✓	✓	✓	35.12	[XX.XX]

In Table 4, adding (L_{cons}) to (L_{pred}) improves FID-10K from **42.73 to 38.85**. The new FID-50K results show the same trend, while the direct comparison between Full LARA and the variant without consistency isolates the contribution of (L_{cons}) . Together with the gap-wise diagnostic in Table 16, these results support transition consistency as an effective mechanism for learning long-range layer transitions.

3. Timestep labeling and valid layer-pair range

Question:

- In Figure 2, the ordering of timesteps should be reversed, right? Lrge t, large noise, bad feature map?
- For the layer-pair sampling strategy in Table 5, could the authors clarify the exact valid range for sampled pairs? For example, is there any lower bound on the source layer (a) or upper bound on the target layer (b)?

The timestep labels in Figure 2 are inconsistent with the noise convention used in the experiment. This is a labeling issue only; the underlying activations and experimental results are unaffected. We will correct the ordering and label the axis directly by noise level and denoising direction.
 For a transformer with (L) blocks, valid source–target pairs satisfy

$$1 \leq a < b \leq L.$$

There are no additional hard bounds on (a) or (b) . All valid pairs remain eligible, while their sampling probabilities depend on the selected pair-sampling schedule.

4. Extension to external alignment methods such as REPA

Weakness:

The paper could discuss whether the layer-aware idea extends to external alignment methods such as REPA. REPA also requires choosing which layer to align to external features, so a natural question is whether LARA-style layer-aware prediction or sampling could reduce this manual choice in external representation alignment as well.

We appreciate this suggestion and agree that the layer-aware mechanism could be extended to improve external alignment methods such as REPA, particularly by reducing the need to manually select the DiT layer aligned with external features.
Table 6 of the paper already shows that LARA and REPA are complementary when trained jointly, improving over LARA alone. However, this experiment combines the two objectives and does not yet apply layer-aware prediction or sampling directly to REPA’s external alignment.
 Extending the shared predictor to map multiple sampled DiT layers into the external representation space is a promising direction that directly follows the reviewer’s suggestion.